3. Estado del arte o de la práctica

Esta sección identifica, caracteriza y analiza seis alternativas para asegurar la calidad de los incrementos producidos por agentes de software. Las alternativas A a E proceden de ecosistemas o prácticas externos a BMAD; la alternativa F, TEA, comparte el ecosistema anfitrión. La comparación examina los mecanismos disponibles y las limitaciones frente al problema de la Sección 1.2. La propuesta ocupa la columna G de la Tabla 3.1. Sus capacidades se presentan como objetivos de diseño que requieren validación, no como resultados experimentales ya demostrados.

3.1 Alternativa A — Middleware de LangChain/LangGraph y evaluadores de LangSmith

Descripción. El ecosistema LangChain proporciona, en su capa de orquestación de grafos, mecanismos de intercepción que permiten insertar lógica antes y después de cada invocación del modelo o de cada llamada a herramienta, así como aprobar o rechazar acciones mediante intervención humana en puntos definidos del grafo. Complementariamente, LangSmith ofrece evaluadores —basados en reglas, en heurísticas o en un modelo juez— que puntúan las salidas de una ejecución contra un conjunto de datos de referencia, con el propósito de detectar regresiones entre versiones de la aplicación agéntica [19].

Por qué no resuelve el problema planteado. La unidad de análisis es incorrecta para el dominio del proyecto. Estos mecanismos gobiernan la invocación —un mensaje, una llamada a herramienta, una traza de ejecución— y no la unidad de trabajo de ingeniería, que es la historia de usuario en su recorrido de estados a través del repositorio. Un evaluador de LangSmith puede establecer que una respuesta particular fue deficiente, pero no puede establecer que la historia 4.2 quedó sin política de seguridad a nivel de fila, porque esa afirmación exige inspeccionar artefactos persistentes del repositorio y no la traza de una sesión. En segundo lugar, la evaluación es diagnóstica y no bloqueante en el sentido exigido: produce puntuaciones que informan a un observador humano fuera de línea, sin condicionar la transición de estado del trabajo. En tercer lugar, la adopción exigiría reimplementar el flujo de BMAD como grafo de LangGraph, lo que contradice la restricción de no modificar el núcleo del framework anfitrión y anularía la transferibilidad del resultado a los equipos que ya lo utilizan.

3.2 Alternativa B — MetaGPT: rol de QA y verificación por procedimiento operativo estandarizado

Descripción. MetaGPT codifica procedimientos operativos estandarizados en secuencias de prompts y distribuye el trabajo entre roles especializados sobre una metáfora de línea de ensamblaje, incluyendo un rol de ingeniero de QA. El framework incorpora además un mecanismo de retroalimentación ejecutiva que depura y ejecuta código en tiempo de ejecución, con mejoras reportadas sobre los benchmarks HumanEval y MBPP [4]. La verificación intermedia por parte de agentes con conocimiento de dominio es un principio explícito de diseño del sistema.

Por qué no resuelve el problema planteado. El contrato de calidad está codificado dentro de los prompts de los roles y no es un artefacto declarativo que el equipo del proyecto pueda declarar, versionar y auditar. En consecuencia, la calidad depende de la formulación de un prompt genérico y no del acuerdo específico de un equipo sobre un producto concreto; no existe la noción de perfil de DoD diferenciado por tipología de historia. En segundo lugar, la verificación se orienta a la corrección funcional —la retroalimentación ejecutiva mide si el código ejecuta y pasa pruebas— y no a las propiedades no funcionales transversales, que constituyen el núcleo del problema estudiado. En tercer lugar, la evaluación del framework se ha realizado sobre benchmarks de generación de funciones aisladas, contexto en el que las categorías de defecto analizadas en este trabajo —RLS, i18n, ciclo de vida de componentes, zonas horarias— no se manifiestan. Finalmente, el rol de QA emite un juicio consultivo: no dispone de autoridad estructural para bloquear la promoción del incremento.

3.3 Alternativa C — AutoGen: agentes conversables y patrón de agente crítico

Descripción. AutoGen propone la abstracción de agente conversable y un conjunto de patrones de conversación configurables que permiten componer sistemas multiagente, entre ellos el patrón crítico-generador, en el que un agente revisa y cuestiona la salida de otro antes de darla por válida [5]. El framework admite además la participación humana en el bucle y la ejecución de código en entornos controlados.

Por qué no resuelve el problema planteado. El patrón crítico introduce revisión pero no gobernanza. La distinción es sustantiva: la crítica es un juicio probabilístico emitido por un modelo sobre otro modelo, mientras que la gobernanza requiere la evaluación determinista de un predicado sobre evidencia observable. Un agente crítico puede omitir una falla, puede aceptar una justificación plausible del generador y, de manera crítica para el problema estudiado, comparte con el generador el sesgo estructural que origina la omisión: si los criterios de aceptación de la historia no mencionan internacionalización, el crítico tampoco tiene motivo para exigirla. La ausencia de un contrato externo declarado convierte la conversación en un sistema cerrado que reproduce sus propias omisiones. Adicionalmente, la conversación es efímera y no deja un artefacto de verificación auditable asociado a la unidad de trabajo, por lo que no satisface el requisito de trazabilidad.

3.4 Alternativa D — SWE-agent, OpenHands y bucles nativos de ejecución de pruebas

Descripción. SWE-agent aborda la resolución autónoma de incidencias reales de repositorios mediante una interfaz agente-computador que estructura las acciones disponibles para el modelo, y demuestra que el diseño de esa interfaz determina de manera significativa el desempeño [6]. La evaluación se realiza sobre SWE-bench, conjunto de tareas construido a partir de incidencias reales de GitHub con verificación automática basada en ejecución [7]. OpenHands generaliza esta arquitectura como plataforma abierta para agentes desarrolladores generalistas, con ejecución en entorno controlado y acceso a terminal, editor y navegador [18].

Por qué no resuelve el problema planteado. El criterio de éxito de estos sistemas es la satisfacción de una suite de pruebas preexistente y externa a la unidad de trabajo. Esta suposición no se sostiene en desarrollo greenfield, que es el escenario dominante en los

proyectos que emplean BMAD: cuando la funcionalidad es nueva, la prueba que la verificaría no existe, y quien debe producirla es el mismo agente que implementa. El bucle de ejecución, por tanto, garantiza consistencia entre código y pruebas, pero no garantiza que las pruebas expresen el comportamiento correcto ni que cubran las propiedades no funcionales. La ausencia de una disciplina test-first impuesta reproduce exactamente el patrón de pruebas frágiles descrito en la Sección 2.2. Adicionalmente, estos sistemas no incorporan noción alguna de adherencia a diseños UI/UX, dado que su dominio de evaluación es la corrección de programas verificada por ejecución, y no operan sobre un ciclo de vida de estados de historia que pudiera interceptarse.

3.5 Alternativa E — Compuertas de calidad estáticas en CI/CD: SonarQube, CodeQL, protección de ramas

Descripción. La práctica industrial establecida instrumenta la calidad mediante análisis estático en la canalización de integración continua. Herramientas como SonarQube evalúan el código incorporado contra umbrales configurables de cobertura, duplicación y densidad de incidencias, y CodeQL ejecuta consultas semánticas sobre representaciones del código para detectar patrones de vulnerabilidad. La protección de ramas convierte el resultado de estas verificaciones en una condición bloqueante para la incorporación de cambios.

Por qué no resuelve el problema planteado. Esta alternativa es la más cercana al mecanismo propuesto y, por ello, la que exige la distinción más precisa. Su limitación es de ubicación temporal y de expresividad. Respecto de la ubicación, la compuerta opera aguas abajo de la generación: el código ya fue escrito, la historia ya fue declarada terminada por el agente y el costo de corrección ya se incurrió; la compuerta detecta, pero no previene. Respecto de la expresividad, el análisis estático verifica propiedades sintácticas y de flujo de datos, pero es estructuralmente incapaz de evaluar criterios que dependen de la intención declarada del equipo: no puede determinar si la política de RLS implementada corresponde al modelo de autorización acordado, si las cadenas visibles fueron externalizadas al catálogo de traducción correcto, ni si el componente implementado corresponde al diseño aprobado en la herramienta de diseño. Finalmente, opera sobre el conjunto de cambios y no sobre la historia, por lo que no produce trazabilidad entre el criterio de calidad acordado y la unidad de trabajo que debía satisfacerlo. La solución propuesta no sustituye estas herramientas: se ubica aguas arriba de ellas y las complementa.

3.6 Alternativa F — BMAD TEA

Descripción. TEA (Test Engineering Architect) es un módulo de BMAD con licencia MIT que ofrece planificación de pruebas, automatización, revisión, evaluación de requisitos no funcionales y trazabilidad con decisión de compuerta. La distribución 2.0.2 inspeccionada incluye ejecución de los escenarios de la fase roja, una CLI de trazabilidad y un hook de control de escritura [21]. Por tanto, su alcance no se limita a listas de chequeo o recomendaciones.

Mecanismos comprobados. El componente atdd-red-check.js ejecuta los escenarios y conserva un reporte; la evaluación de si fallan por el comportamiento esperado corresponde a la capa de evaluación que lo invoca. La CLI tea-trace puede producir un código de salida distinto de cero cuando la compuerta resulta FAIL. El hook tea-enforce.cjs puede bloquear

escrituras de pruebas por un subconjunto de reglas mecánicamente verificables. Asimismo, el flujo Trace admite alcance de historia y produce evidencia vinculada a criterios, por lo que se reconoce cobertura de D6.

Brecha frente a la propuesta. En la distribución examinada no se identificó un contrato de DoD del equipo parametrizado por tipología de historia que combine criterios de producto, consulta del diseño aprobado y reglas para la transición de la historia a revisión. Los controles de TEA inspeccionados se concentran en calidad de pruebas y decisiones de liberación; la existencia de esos controles no equivale por sí sola a interceptar la transición de estado de cualquier historia del proyecto. La brecha investigable se delimita a esa integración y al contrato específico del equipo. Debe demostrarse mediante una comparación ejecutable y no presumirse por la ausencia de una opción en la documentación.

Relación entre los artefactos. La propuesta puede utilizar TEA para producir pruebas, recopilar evidencia o decidir compuertas, y añadir la adaptación al ciclo de vida de historias y los perfiles de DoD del proyecto. Su aporte potencial es la integración y su evaluación empírica en el contexto definido. No se sostiene una afirmación general de que TEA carece de ejecución, trazabilidad o mecanismos bloqueantes.

Alcance de la revisión. Se inspeccionó el paquete npm bmad-method-test-architecture-enterprise, versión 2.0.2, el 14 de octubre de 2026. Se verificó su integridad SHA-512 y se registraron los archivos usados en la comparación. Los resultados se circunscriben a esa distribución; las funciones de otras versiones y la suficiencia de la brecha se contrastarán con el tutor.

3.7 Matriz de incidencia comparativa

D1 — ATDD mandatorio antes del código: El flujo condiciona la generación de código de producción a una prueba de aceptación ejecutada que falle por el comportamiento faltante esperado.

D2 — Perfiles de DoD por tipología: El contrato del equipo es un artefacto externo, versionado y parametrizable por tipología de historia.

D3 — Consulta verificada del diseño UI/UX: Se exige y registra la consulta del diseño aprobado antes de implementar componentes visuales.

D4 — Extensión sin modificar el núcleo: El mecanismo puede incorporarse sin modificar el código del framework anfitrión.

D5 — Bloqueo de la transición de historia: El incumplimiento del contrato impide la transición de la historia al estado de revisión o terminada.

D6 — Evidencia auditable por historia: Se produce un artefacto persistente que vincula criterios con evidencia concreta para una historia.

Dimensión	A. LangChain /	B. MetaGPT	C. AutoGen	D. SWE-agent / OpenHa	E. Compuertas	F. BMAD TEA 2.0.2	G. Governance

	LangSmith			nds	CI/CD		Layer (objetivo)
D1 ATDD mandatorio antes del código	No	No	No	Parcial	No	Parcial	Sí
D2 Perfiles de DoD por tipología	No	No	No	No	Parcial	Parcial	Sí
D3 Consulta verificada del diseño UI/UX	No	No	No	No	No	No	Sí
D4 Extensión sin modificar el núcleo	No	No	Parcial	No	Sí	Sí	Sí
D5 Bloqueo de la transición de historia	No	No	No	Parcial	Parcial	Parcial	Sí
D6 Evidencia auditable por historia	Parcial	No	No	Parcial	Parcial	Sí	Sí

Cobertura cualitativa de las dimensiones de comparación

	A LangChain	B MetaGPT	C AutoGen	D SWE / OH	E CI/CD	F TEA 2.0.2	G Propuesta*
D1 ATDD mandatorio antes del código	No	No	No	Parcial	No	Parcial	Sí
D2 Perfiles de DoD por tipología	No	No	No	No	Parcial	Parcial	Sí
D3 Consulta verificada del diseño UI/UX	No	No	No	No	No	No	Sí
D4 Extensión sin modificar el núcleo	No	No	Parcial	No	Sí	Sí	Sí
D5 Bloqueo de la transición de historia	No	No	No	Parcial	Parcial	Parcial	Sí
D6 Evidencia auditable por historia	Parcial	No	No	Parcial	Parcial	Sí	Sí

Escala: No = 0 · Parcial = 0,5 · Sí = 1. Un tono más cálido indica mayor cobertura de la dimensión.

* G representa objetivos de diseño pendientes de validación. La escala no mide eficacia ni superioridad.

Escala cualitativa: No=0; Parcial=0,5; Sí=1. La columna G representa objetivos de diseño pendientes de validación.

Síntesis del análisis. Ninguna de las seis alternativas examinadas satisface de forma demostrada la combinación completa de dimensiones en el contexto del proyecto. TEA es la alternativa más próxima por compartir BMAD y ofrecer ejecución de pruebas, trazabilidad y mecanismos de compuerta. La brecha no es la inexistencia de controles de calidad, sino la integración de un contrato de DoD del equipo, diferenciado por tipología, con la consulta de diseño y las transiciones del ciclo de vida de las historias. La columna G expresa la cobertura que se pretende construir y evaluar. La matriz es una comparación cualitativa de mecanismos, no una medición de reducción de defectos ni una prueba de superioridad de la propuesta.

Referencia incorporada

[21] BMad Code, “Test Engineering Architect (TEA), versión 2.0.2”, código fuente de software, licencia MIT, 2026. [En línea]. Disponible: https://www.npmjs.com/package/bmad-method-test-architecture-enterprise/v/2.0.2. Consultado: 14 de octubre de 2026. Archivos inspeccionados: README.md; cli/atdd-red-check.js; cli/trace.js; skills/bmad-testarch-framework/resources/hooks/tea-enforce.cjs; skills/bmad-testarch-trace/trace-template.md.